SymPy Bug Report

1 Bug 37: Wrong Sign in an Antiderivative on the Negative Real Branch

1.1 Minimal Reproducer

```
from sympy import N, diff, integrate, sqrt, symbols

x = symbols("x")
integrand = 1/(x*sqrt(x**2 - 1))
F = integrate(integrand, x)
dF = diff(F, x)

print("antiderivative =", F)
print("derivative =", dF)
print("integrand at x=-3:", N(integrand.subs(x, -3), 50))
print("derivative at x=-3:", N(dF.subs(x, -3), 50))
print("difference:", N((dF - integrand).subs(x, -3), 50))
```

1.2 Observed Behavior

```
antiderivative = Piecewise((I*acosh(1/x), 1/Abs(x**2) > 1), (-asin(1/x), True))
derivative = Piecewise((-I/(x**2*sqrt(-1 + 1/x)*sqrt(1 + 1/x)), 1/Abs(x**2) > 1), (1/(x
    **2*sqrt(1 - 1/x**2)), True))
integrand at x=-3: -0.11785113019775792073347406035080817321413932294808
derivative at x=-3: 0.11785113019775792073347406035080817321413932294808
difference: 0.23570226039551584146694812070161634642827864589616
```

The returned expression differentiates to the wrong real sign at the ordinary point $x = -3$. The residual is not zero: the derivative of SymPy's returned antiderivative is the positive opposite of the requested integrand on this negative real branch.

1.3 Expected Behavior

For an indefinite integral returned as an antiderivative, differentiating the result should recover

$$\frac{1}{x\sqrt{x^2-1}}$$

at ordinary points of the selected real branch. In particular, at $x = -3$ the derivative should evaluate to

$$-0.11785113019775792073347406035080817321413932294808\ldots,$$

matching the integrand rather than its negation.

1.4 Mathematical Explanation

On the real interval $x < -1$, the principal square root satisfies $\sqrt{x^2 - 1} > 0$, while $x < 0$. Therefore

$$\frac{1}{x\sqrt{x^2 - 1}} < 0.$$

SymPy's fallback branch is $-\operatorname{asin}(1/x)$. Differentiating it gives

$$\frac{d}{dx}\left[-\operatorname{asin}\left(\frac{1}{x}\right)\right] = \frac{1}{x^2\sqrt{1 - 1/x^2}}.$$

For $x < -1$, this derivative is positive, because both x^2 and $\sqrt{1 - 1/x^2}$ are positive. It is related to the target integrand by

$$\frac{1}{x^2\sqrt{1 - 1/x^2}} = \frac{1}{|x|\sqrt{x^2 - 1}},$$

which equals the requested integrand on $x > 1$ but equals its negation on $x < -1$. The discrepancy is therefore not a harmless additive constant, simplification choice, or equivalent branch representation: the derivative has the wrong sign on a nonsingular real interval.

1.5 Source-Level Diagnosis

The diagnosis narrows the source of the bad result to the Meijer-G indefinite integration path in `sympy/integrals/meijerint.py`, around lines 1653–1777. The relevant functions named in the diagnosis are:

```
meijerint_indefinite
_meijerint_indefinite_1
```

The public call to `integrate` constructs an `Integral` and reaches `Integral._eval_integral` in `sympy/integrals/integrals.py`. The ordinary Risch and heuristic paths do not handle this algebraic integrand, so the Meijer-G fallback calls `meijerint_indefinite`; direct probing of that function reportedly produces the same bad `Piecewise` result.

The diagnosed path rewrites the integrand as a Meijer-G expression, applies a generic antiderivative formula, expands the result with `hyperexpand`, combines powers using `powdenest(..., polar=True)`, and then removes polar or branch annotations with `_my_unpolarify(_clean(...))`. The important outcome is that the final fallback branch is `-asin(1/x)` under a `True` condition, after an inner branch guarded by `1/Abs(x**2) > 1`. That fallback conflates the two exterior real intervals $x > 1$ and $x < -1$.

The source-level behavior causes the observed mathematical failure because $-\operatorname{asin}(1/x)$ is a correct real antiderivative on the positive exterior branch but has the opposite derivative sign on the negative exterior branch. The diagnosis therefore identifies a branch/sign loss during the Meijer/hyperexpanded expression's conversion back to ordinary inverse trigonometric functions. A plausible fix direction supported by the diagnosis is to retain a sign-dependent `Piecewise` result for $x > 1$ and $x < -1$, or to decline this simplified inverse-trigonometric antiderivative when the branch cannot be made valid over the whole fallback condition.

1.6 Additional Failing Instances

The same failure appears across representative negative real sample points in the interval $x < -1$. The parametrized verification checks $x = -2, -3, -4, -5, -6, -7$. In each case, the returned

fallback branch differentiates to $1/(|x|\sqrt{x^2-1})$, while the requested integrand is $1/(x\sqrt{x^2-1})$; since $x<0$, the two values have opposite signs.

Input / Expression	SymPy behavior	Expected behavior
$x=-2,\ F'(-2)$	$+1/(2\sqrt{3})$	$-1/(2\sqrt{3})$
$x=-3,\ F'(-3)$	$+1/(6\sqrt{2})$	$-1/(6\sqrt{2})$
$x=-4,\ F'(-4)$	$+1/(4\sqrt{15})$	$-1/(4\sqrt{15})$
$x=-5,\ F'(-5)$	$+1/(10\sqrt{6})$	$-1/(10\sqrt{6})$
$x=-6,\ F'(-6)$	$+1/(6\sqrt{35})$	$-1/(6\sqrt{35})$
$x=-7,\ F'(-7)$	$+1/(28\sqrt{3})$	$-1/(28\sqrt{3})$

1.7 Related Issues Assessment

The bug-finding harness performed a best-effort deduplication check and classified this as a new instance of a known failure mode. The closest public precedents in the same $\sqrt{x^2-1}$ integration family are two open/closed SymPy issues: sympy/sympy#20982 (closed), "integrate(1 / (x ** 2 * sqrt(x ** 2 - 1))) gives wrong integral (lost sign when $x<0$)", which reports the same negative-real sign-loss symptom on a sibling $1/(x^2\sqrt{x^2-1})$ integrand; and sympy/sympy#23566 (open), "integrate(1/sqrt(x**2-1)) shouldn't be acosh(x)", which concerns the same Meijer-G acosh / asin `Piecewise` branch handling for $\sqrt{x^2-1}$. No public issue or pull request, however, targets the exact $1/(x\sqrt{x^2-1})$ indefinite integral having the wrong sign on negative real inputs. The broad branch-cut family is therefore known and publicly documented, while this minimized negative-branch counterexample is specific and previously unreported, remaining individually actionable as an issue and regression test.

1.8 Proposed SymPy Regression Test

```
import pytest
from sympy import N, diff, integrate, sqrt, symbols


@pytest.mark.parametrize("x_value", [-2, -3, -4, -5, -6, -7])
def test_integral_of_inverse_x_sqrt_x2_minus_1_negative_branch(x_value):
    x = symbols("x")
    integrand = 1/(x*sqrt(x**2 - 1))
    antiderivative = integrate(integrand, x)
    residual = diff(antiderivative, x) - integrand
    assert abs(complex(N(residual.subs(x, x_value), 50))) < 1e-40
```